

GrapesJS TABANLI HEADLESS CMS VE PORTFOLYO SİSTEMİ

Teknik Mimari ve Sistem Tasarımı Dokümantasyonu
v2.0 — Genişletilmiş ve Guncellenme Edition

Hazırlayan	Mehmet Akalin
Versiyon	2.0 — Kapsamlı Revizyon
Tarih	Mayıs 2026
Durum	Aktif Geliştirme

Bu Doküman Hakkında

Bu teknik mimari dokümanı, v1.0 tasarımının kapsamlı bir revizyonudur. Orijinal tasarımın güçlü yönleri korunarak medya yönetimi, arama altyapısı, güvenlik iyileştirmeleri, rate limiting ve genişletilmiş veri modelleri gibi eksik bileşenler sisteme dahil edilmiştir.

İçindekiler

1. Giriş ve Proje Özeti

Bu doküman, modern web standartlarına uygun ayrıştırılmış (decoupled) bir mimariyle geliştirilmesi planlanan kişisel portfolyo ve dinamik içerik yönetim sistemi (Headless CMS) projesinin teknik altyapısını ve sistem tasarımını tanımlamaktadır.

Projenin temel amacı, klasik statik blog veya hazır içerik yönetim sistemlerinin (WordPress vb.) aksine, geliştiriciye tam mizanpaj özgürlüğü tanıyan GrapesJS görsel sayfa oluşturucusunu özel bir mimariyle entegre etmektir. Bu sayede sistemin hem portföy vitrin işlevi hem de tam özellikli bir CMS işlevi görmesi hedeflenmektedir.

1.1. Temel Hedefler

- Geliştirici deneyimi: GrapesJS ile sürükle-bırak tasarım özgürlüğü
- Performans: Ziyaretçilere önceden derlenmiş HTML sunarak sıfır JS render yükü
- Genişletilebilirlik: Mobil uygulama veya farklı frontend'ler için hazır API katmanı
- Güvenlik: Uçtan uca XSS koruması, JWT kimlik doğrulama, CORS kısıtlaması
- SEO: Slug tabanlı URL yapısı, metadata yönetimi, statik render avantajı
- Sürdürülebilirlik: Docker tabanlı deploy, ortam değişkeni yönetimi, test altyapısı

1.2. v1.0'dan v2.0'a Değişiklikler

Bu revizyon, orijinal tasarımdaki şu eksiklikleri giderir:

- **Medya yönetimi:** Görsel yükleme, depolama ve optimizasyon altyapısı eklendi
- **Arama altyapısı:** PostgreSQL full-text search ve kategori/etiket sistemi eklendi
- **Rate limiting:** Brute force koruması ve API kota yönetimi eklendi
- **Refresh token rotasyonu:** Güvenli token yenileme akışı belgelendi
- **İki katmanlı XSS koruması:** Backend sanitize'a ek frontend güvenlik katmanı eklendi
- **Drag-and-drop sıralama:** Manuel sort_order yerine API destekli sıralama eklendi
- **Test stratejisi:** Unit, entegrasyon ve E2E test planı eklendi
- **CI/CD pipeline:** GitHub Actions tabanlı deploy otomasyonu belgelendi

2. Genel Sistem Mimarisi

Proje, istemci ve sunucu katmanlarının tamamen bağımsız çalıştığı Headless (Başsız) Mimari modelini benimser. Bu yapı, gelecekte mobil uygulamalar veya farklı ön yüz arayüzleri eklendiğinde arka uç servislerinin hiç değiştirilmeden yeniden kullanılabilmesini sağlar.

2.1. İstemci Katmanı (Frontend SPA)

Bileşen	Detay
Framework	React 18 + TypeScript 5.x
Stil sistemi	Tailwind CSS v3
Editör	GrapesJS Core (son stabil versiyon)
HTTP istemcisi	Axios — interceptor ile token yenileme
State yönetimi	React Context API veya Zustand (hafif)
Router	React Router v6 — lazy loading ile
Güvenlik	DOMPurify — frontend XSS sanitize

2.2. Sunucu Katmanı (Backend API)

Bileşen	Detay
Framework	Django 5.x
REST katmanı	Django REST Framework (DRF)
Kimlik doğrulama	djangorestframework-simplejwt — Access + Refresh token
XSS temizliği	nh3 (Python) — bleach artık deprecated
Rate limiting	django-ratelimit veya DRF throttling
Medya depolama	django-storages + AWS S3 (prod) / yerel disk (dev)
Görsel optimizasyon	Pillow — yükleme sırasında WebP dönüşümü ve boyut kısıtlama
Görev kuyruğu	Celery + Redis — async görsel işleme (opsiyonel)

2.3. Veritabanı Katmanı

Ortam	Veritabanı ve Gerekçe
Production	PostgreSQL — JSONField, full-text search ve gelişmiş indeksleme için zorunlu
Development	SQLite — hafif, sıfır konfigürasyon, yerel geliştirme için yeterli

Test

SQLite in-memory — hızlı test koşulları için, CI ortamında da kullanılabilir

Kritik Tasarım Kararı: İkili İçerik Saklama

GrapesJS iki farklı formatta çıktı üretir: HTML/CSS (ziyaretçi render için) ve JSON (editör ağaç yapısı için). Sistem bu ikisini aynı anda veritabanında tutar. Bu sayede ziyaretçiye hızlı statik HTML sunulurken admin panelinde editör eksiksiz yeniden yüklenebilir. Performans ile editör esnekliği arasındaki en verimli denge bu şekilde sağlanmaktadır.

3. Veri Modelleri ve Veritabanı Şeması

Django ORM üzerinde kurgulanan veritabanı mimarisi, ilişkisel bütünlüğü korurken büyük veri bloklarının performans kaybı yaşatmadan saklanması amaçlar. v2.0 ile birlikte medya yönetimi, kategori/etiket sistemi ve gelişmiş sıralama modelleri eklenmiştir.

3.1. BlogPost Modeli

Alan Adı	Veri Tipi	Açıklama
id	UUIDField (PK)	Benzersiz kimlik — sıralı integer yerine UUID ile güvenlik
title	CharField(255)	Makale başlığı
slug	SlugField(unique=True)	SEO uyumlu URL uzantısı — otomatik üretim + manuel override
summary	TextField	Liste sayfalarında gösterilen özet; max 300 karakter önerilir
content_html	TextField	Ziyaretçiye render edilen saf HTML/CSS — backend'de nh3 ile sanitize edilmiş
content_json	JSONField	GrapesJS editör ağaç yapısı — yalnızca admin paneli kullanır
status	CharField(choices)	DRAFT / PUBLISHED / ARCHIVED — üçlü durum sistemi
featured_image	ForeignKey(Media)	YENİ: Kapak görseli — Media modeline FK
categories	M2M(Category)	YENİ: Çok-çok ilişki ile kategori ataması
tags	M2M(Tag)	YENİ: Etiket sistemi — filtreleme ve arama için
meta_title	CharField(60, blank)	YENİ: SEO meta başlığı — boşsa title kullanılır
meta_description	TextField(blank)	YENİ: SEO meta açıklaması — max 160 karakter
reading_time	IntegerField	YENİ: Tahmini okuma süresi (dakika) — kayıta otomatik hesaplama
view_count	PositiveIntegerField	YENİ: Görüntülenme sayısı — rate limited endpoint ile artırma
created_at	DateTimeField(auto)	Oluşturulma zaman damgası
updated_at	DateTimeField(auto)	Son güncellenme zaman damgası

3.2. Project Modeli

Alan Adı	Veri Tipi	Açıklama
----------	-----------	----------

id	UUIDField (PK)	Benzersiz kimlik
title	CharField(150)	Proje adı (örn: AUtomotion-R)
slug	SlugField(unique)	YENİ: SEO uyumlu URL slug
description	TextField	Teknik detaylar, kapsam ve başarılar
tech_stack	JSONField	Teknoloji listesi — örn: ["ROS2", "SLAM", "Django"]
thumbnail	ForeignKey(Media)	YENİ: Proje kapak görseli — Media modeline FK
gallery	M2M(Media)	YENİ: Proje galeri görselleri
github_url	URLField(blank)	Açık kaynak depo bağlantısı
live_url	URLField(blank)	Canlı önizleme adresi
status	CharField(choices)	YENİ: ACTIVE / ARCHIVED / IN_PROGRESS
is_featured	BooleanField	YENİ: Ana sayfada öne çıkarma bayrağı
sort_order	IntegerField(default=0)	Listelenme sırası — PATCH /reorder/ endpoint'i ile API üzerinden güncellenir
start_date	DateField(blank)	YENİ: Proje başlangıç tarihi
end_date	DateField(blank)	YENİ: Proje bitiş tarihi (devam ediyorsa boş)

3.3. Media Modeli (YENİ)

v1.0'da eksik olan medya yönetim altyapısı bu model ile tamamlanmaktadır. Blog görselleri, proje kapakları ve galeri resimleri bu merkezi tablo üzerinden yönetilir.

Alan Adı	Veri Tipi	Açıklama
id	UUIDField (PK)	Benzersiz kimlik
file	FileField	Fiziksel dosya — S3'te yükleme_yolu/dosya_adi formatında
original_name	CharField(255)	Orijinal dosya adı — kullanıcı bilgisi için saklanır
mime_type	CharField(100)	image/webp, image/jpeg gibi MIME tipi
file_size	PositiveIntegerField	Bayt cinsinden dosya boyutu
width	PositiveIntegerField	Piksel genişliği — Pillow ile otomatik tespit
height	PositiveIntegerField	Piksel yüksekliği
alt_text	CharField(255, blank)	Erişilebilirlik ve SEO için alternatif metin
uploaded_at	DateTimeField(auto)	Yükleme zaman damgası

3.4. Category ve Tag Modelleri (YENİ)

Blog yazılarını organize etmek ve arama/filtreleme altyapısını beslemek için iki ayrı model tanımlanmıştır.

Alan Adı	Kategori/Etiket	Açıklama
id	Her ikisi	UUIDField primary key
name	Her ikisi	Görüntülenen ad
slug	Her ikisi	URL uyumlu benzersiz kimlik
description	Sadece Category	Kategori açıklaması
color	Sadece Tag	Hex renk kodu — frontend badge rengi için

4. API Tasarımı ve Uç Noktalar

Django REST Framework kullanılarak tasarlanan API mimarisi RESTful prensiplerine tam uyumludur. JWT tabanlı kimlik doğrulama, rate limiting ve sayfalama standart olarak uygulanmaktadır.

4.1. Kimlik Doğrulama Uç Noktaları

Metot	Endpoint	İzin	Açıklama
POST	/api/token/	Herkese Açık	Kullanıcı adı + şifre ile Access/Refresh token çifti üretir
POST	/api/token/refresh/	Herkese Açık	Refresh token ile yeni Access token üretir — token rotasyonu uygulanır
POST	/api/token/blacklist/	Admin (JWT)	Logout — Refresh token'ı geçersiz kılar

4.2. Blog Uç Noktaları

Metot	Endpoint	İzin	Açıklama
GET	/api/posts/	Herkese Açık	Yayınlanmış yazılar — sayfalama, kategori/etiket filtresi, full-text arama
GET	/api/posts/<slug>/	Herkese Açık	Detay sayfası — content_html, meta, ilgili yazılar döner
GET	/api/posts/<slug>/related/	Herkese Açık	YENİ: Aynı kategoriden ilgili yazılar (max 3)
POST	/api/posts/<slug>/view/	Herkese Açık	YENİ: Görüntülenme sayısını artırır — 1 IP'den günde 1 kez (rate limited)
GET	/api/admin/posts/	Admin (JWT)	Tüm durumlar dahil tam liste — DRAFT, PUBLISHED, ARCHIVED
POST	/api/admin/posts/	Admin (JWT)	GrapesJS HTML + JSON ile yeni içerik oluşturur
PUT	/api/admin/posts/<id>/	Admin (JWT)	Tam güncelleme
PATCH	/api/admin/posts/<id>/	Admin (JWT)	Kısmi güncelleme — sadece status değişimi için önerilir
DELETE	/api/admin/posts/<id>/	Admin (JWT)	Soft delete — status=ARCHIVED yapar, fiziksel silme yapmaz

4.3. Proje ve Medya Uç Noktaları

Metot	Endpoint	İzin	Açıklama
-------	----------	------	----------

GET	/api/projects/	Herkese Açık	Aktif projeleri sort_order'a göre listeler
GET	/api/projects/<slug>/	Herkese Açık	YENİ: Proje detayı — galeri ve tech_stack dahil
PATCH	/api/admin/projects/reorder/	Admin (JWT)	YENİ: Proje sıralamasını günceller — [{id, sort_order}] listesi alır
GET	/api/categories/	Herkese Açık	YENİ: Yazı sayısı ile birlikte kategori listesi
GET	/api/tags/	Herkese Açık	YENİ: Etiket listesi
POST	/api/admin/media/	Admin (JWT)	YENİ: Görsel yükleme — multipart/form-data, max 5MB, WebP dönüşümü
GET	/api/admin/media/	Admin (JWT)	YENİ: Medya kütüphanesi listesi — GrapesJS asset manager entegrasyonu için
DELETE	/api/admin/media/<id>/	Admin (JWT)	YENİ: Fiziksel dosya ve veritabanı kaydı silinir

4.4. API Genel Kuralları

- Sayfalama: Tüm liste endpoint'lerinde ?page=N&page_size=10 parametresi desteklenir
- Arama: /api/posts/?search=kelime — PostgreSQL full-text search ile title, summary, tag alanlarında
- Filtreleme: ?category=slug&tag=isim&status=PUBLISHED kombinasyonları desteklenir
- Sıralama: ?ordering=-created_at (azalan) veya ?ordering=sort_order
- Yanıt formatı: Tüm endpoint'ler application/json döner; medya yüklemesi multipart/form-data kabul eder

5. GrapesJS Entegrasyonu ve Veri Akış Yaşam Döngüsü

5.1. Editörün Başlatılması

React bileşeni yüklendiğinde (component mount) GrapesJS motoru hedef bir element üzerinde tetiklenir. Editöre özel teknik bloklar, medya yöneticisi ve uyarı kutuları enjekte edilir.

```
// GrapesJS React entegrasyonu - useRef ile DOM element bağlama
const editorRef = useRef(null);
useEffect(() => {
  const editor = grapesjs.init({
    container: editorRef.current,
    storageManager: false, // Kendi save pipeline'ımızı kullanıyoruz
    assetManager: {
      assets: [], // API'den dinamik yüklenir
      uploadFile: handleMediaUpload, // /api/admin/media/ endpoint'ine bağlı
    },
  });
  // Özel bloklar ve temizlik
  return () => editor.destroy(); // Bellek sızıntısını önlemek için
}, [1]);
```

5.2. Veri Kayıt Döngüsü (Save Pipeline)

Geliştirici editörde çalışmasını tamamlayıp kaydet butonuna bastığında şu adımlar sırayla çalışır:

1. GrapesJS'ten HTML/CSS ve JSON verisi alınır
2. HTML, frontend'de DOMPurify ile ön sanitize edilir (birinci savunma katmanı)
3. Axios ile Authorization: Bearer <token> header'ı ile Django endpoint'ine gönderilir
4. Django, HTML'i nh3 kütüphanesi ile tekrar sanitize eder (ikinci savunma katmanı)
5. Doğrulan veri PostgreSQL'e kaydedilir; başarı yanıtı frontend'e döner

```
// Save pipeline - çift katmanlı güvenlik
const html = editor.getHtml() + `<style>${editor.getCss()}</style>`;
const json = editor.getComponents();
const cleanHtml = DOMPurify.sanitize(html); // Frontend: 1. katman
await axios.post('/api/admin/posts/', {
  title, slug, summary,
  content_html: cleanHtml,
  content_json: json,
  status: 'DRAFT',
}, { headers: { Authorization: `Bearer ${accessToken}` } });
```

5.3. Token Yenileme Akışı

Access token süresi dolduğunda Axios interceptor otomatik olarak devreye girer ve refresh token ile yeni bir access token alır. Bu işlem kullanıcıya fark ettirmeden arka planda gerçekleşir.

```
// Axios interceptor - otomatik token yenileme
axios.interceptors.response.use(null, async (error) => {
  if (error.response?.status === 401 && !error.config._retry) {
    error.config._retry = true;
    const { data } = await axios.post('/api/token/refresh/', {
      refresh: getRefreshToken(),
    });
    setAccessToken(data.access);
    error.config.headers.Authorization = `Bearer ${data.access}`;
    return axios(error.config); // Başarısız isteği tekrar dene
  }
  return Promise.reject(error);
});
```

5.4. Ziyaretçi Render Süreci

Ziyaretçi blog detay sayfasına girdiğinde React tarafı yalnızca content_html içeriğini çeker. Bu veri DOMPurify'dan geçirilerek sayfaya enjekte edilir. Ek JavaScript yükü sıfırdır.

```
// Frontend render - DOMPurify ile güvenli HTML enjeksiyonu
const cleanContent = DOMPurify.sanitize(post.content_html, {
  ALLOWED_TAGS: ['p', 'h1', 'h2', 'h3', 'pre', 'code', 'img', 'a', 'ul', 'ol', 'li',
    'blockquote', 'strong', 'em', 'table', 'thead', 'tbody', 'tr', 'td'],
  ALLOWED_ATTR: ['href', 'src', 'alt', 'class', 'target', 'rel'],
  FORCE_HTTPS: true,
});
return <div dangerouslySetInnerHTML={{ __html: cleanContent }} />;
```

6. Güvenlik Mimarisi

Güvenlik Felsefesi

Bu sistem 'derinlemesine savunma' (defense in depth) prensibini benimser. Her tehdit vektörüne karşı birden fazla bağımsız savunma katmanı uygulanır. Tek bir katmanın atlatılması sistemin çökmesine yol açmaz.

6.1. XSS Koruması — İki Katmanlı Savunma

- **1. Katman (Frontend):** DOMPurify ile HTML sanitize — içerik DOM'a yazılmadan önce uygulanır
- **2. Katman (Backend):** Python nh3 kütüphanesi ile veritabanına yazmadan önce sanitize
- Neden nh3, bleach değil: bleach kütüphanesi artık aktif olarak bakım görmemektedir. nh3, Rust tabanlı html5ever üzerine inşa edilmiş modern ve güvenli alternatiftir.
- İzin verilen HTML etiketleri whitelist ile kısıtlanır; script, iframe, form, object ve embed etiketleri her koşulda kaldırılır

```
# Django backend - nh3 ile HTML sanitize
import nh3
ALLOWED_TAGS = {'p', 'h1', 'h2', 'h3', 'pre', 'code', 'img', 'a', 'ul', 'ol', 'li',
                 'blockquote', 'strong', 'em', 'table', 'thead', 'tbody', 'tr', 'td'}
ALLOWED_ATTRS = {'a': {'href', 'target', 'rel'}, 'img': {'src', 'alt', 'class'}}
clean_html = nh3.clean(raw_html, tags=ALLOWED_TAGS, attributes=ALLOWED_ATTRS)
```

6.2. JWT Güvenliği

- **Access token süresi:** 15 dakika — kısa tutulur; sızıntı riskini minimize eder
- **Refresh token süresi:** 7 gün — httpOnly cookie olarak saklanır (localStorage değil)
- **Token rotasyonu:** Her refresh işleminde eski refresh token geçersiz kılınır (blacklist)
- **Algoritma:** HS256 minimum; RS256 tercih edilir (asimetrik — public key güvenli paylaşılabilir)

6.3. Rate Limiting

v1.0'da eksik olan bu bileşen özellikle /api/token/ endpoint'ini korumak için kritiktir.

Endpoint	Limit	Pencere
/api/token/ (login)	5 deneme	15 dakika / IP
/api/token/refresh/	20 istek	1 dakika / kullanıcı
/api/posts/<slug>/view/	1 artırma	24 saat / IP

/api/admin/media/ (yükleme)	20 yükleme	1 saat / kullanıcı
Genel API	200 istek	1 dakika / IP

6.4. Diğer Güvenlik Önlemleri

- **CORS:** CORS_ALLOWED_ORIGINS yalnızca portfolyo domain'ini içerir — geliştirme ve production ayrı konfigürasyon
- **HTTPS:** Üretim ortamında tüm trafik HTTPS zorunludur; HTTP'den otomatik yönlendirme
- **Güvenli başlıklar:** django-security veya django-csp ile Content-Security-Policy, X-Frame-Options, HSTS başlıkları
- **Dosya yükleme:** Yüklenen dosyalar MIME türü doğrulanır, maksimum 5MB ile sınırlandırılır, rastgele isimle kaydedilir
- **SQL injection:** Django ORM her koşulda parametrelili sorgu kullanır — ham SQL yasak
- **DB indeksleri:** slug, status alanları db_index=True ile; tam metin araması için PostgreSQL GIN indeksi

7. Medya Yönetimi (YENİ)

v1.0'ın en kritik eksikliği idi. Blog görselleri ve proje kapakları için uçtan uca bir medya boru hattı tasarlanmıştır.

7.1. Görsel Yükleme Akışı

6. Admin paneli GrapesJS asset manager veya ayrı bir yükleme arayüzünden dosya seçer
7. Frontend, dosyayı /api/admin/media/ endpoint'ine multipart/form-data olarak gönderir
8. Django MIME türünü ve boyutunu doğrular (max 5MB, izin verilen tipler: jpeg, png, webp, gif)
9. Pillow ile görsel işlenir: WebP formatına dönüştürülür, max 2048px boyuta indirgenir
10. Üretimde AWS S3'e, geliştirmede yerel medya dizinine kaydedilir
11. Media kaydı oluşturulur, CDN URL'si ve metadata frontend'e döner

7.2. Depolama Konfigürasyonu

```
# settings.py - ortama göre storage arka ucu
if DEBUG:
    DEFAULT_FILE_STORAGE = 'django.core.files.storage.FileSystemStorage'
    MEDIA_URL = '/media/'
    MEDIA_ROOT = BASE_DIR / 'media'
else:
    DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
    AWS_STORAGE_BUCKET_NAME = env('AWS_STORAGE_BUCKET_NAME')
    AWS_S3_REGION_NAME = env('AWS_S3_REGION_NAME')
    AWS_S3_CUSTOM_DOMAIN = env('AWS_CLOUDFRONT_DOMAIN') # CDN
```

7.3. GrapesJS Entegrasyonu

GrapesJS asset manager, medya kütüphanesinden görsel seçmeyi ve yeni görsel yüklemeyi destekler. Editör açıldığında mevcut medyalar /api/admin/media/ endpoint'inden dinamik olarak yüklenir.

```
// GrapesJS asset manager - API entegrasyonu
assetManager: {
  assets: [],
  upload: '/api/admin/media/',
  uploadName: 'file',
  headers: { Authorization: `Bearer ${accessToken}` },
  autoAdd: true,
},
```

8. Performans Optimizasyonu

8.1. Veritabanı Performansı

Alan / Sorgu	İndeks Türü	Gerekçe
slug	BTree (unique)	URL bazlı lookuplar için zorunlu
status	BTree	Yayınlanmış yazı filtresi her sorguda kullanılır
created_at	BTree	Tarihe göre sıralama ve sayfalama
title + summary	PostgreSQL GIN	Full-text arama — tsvector ile
categories M2M	Otomatik (Django)	Kategori filtresi

8.2. Önbellekleme Stratejisi

- **Sayfa önbelleği:** Ziyaretçi API yanıtları Redis veya veritabanı önbelleği ile 15 dakika saklanır
- **Cache invalidation:** Admin panelinden yazı güncellendiğinde ilgili cache key'i silinir
- **Statik varlıklar:** Vercel CDN ile tüm JS/CSS dosyaları edge'de önbelleklenir
- **Görsel CDN:** AWS CloudFront ile medya dosyaları global edge node'lardan sunulur
- Önbellek başlıkları: Cache-Control: max-age=3600, s-maxage=86400

8.3. Frontend Performansı

- **Lazy loading:** React Router ile sayfa bileşenleri kod bölümlleme (code splitting) ile yüklenir
- **GrapesJS izolasyonu:** Editör yalnızca admin rotasında yüklenir — ziyaretçi paketine dahil edilmez
- **Görsel optimizasyon:** WebP formatı, srcset ile responsive görüntüler, loading='lazy' niteliği
- **Bundle analizi:** webpack-bundle-analyzer ile periyodik paket boyutu denetimi

9. Test Stratejisi (YENİ)

v1.0'da belgelenmemiş olan test altyapısı, yazılım kalitesini güvence altına almak için üç katmanda planlanmıştır.

9.1. Backend Testleri (pytest)

Test Türü	Araç	Kapsam
Unit testler	pytest + pytest-django	Model validasyonları, sanitize fonksiyonları, slug üretimi
API testleri	DRF APITestCase	Tüm endpoint'ler — auth gerektiren ve gerektirmeyen
Güvenlik testleri	pytest	XSS payload'ları, JWT manipülasyonu, rate limit aşımı
Kapsam hedefi	%80+	Kritik path'ler %100 — auth, sanitize, medya yükleme

9.2. Frontend Testleri

- **Unit testler:** Vitest + React Testing Library ile bileşen testleri
- **API mock'ları:** msw (Mock Service Worker) ile backend bağımsız UI testleri
- **E2E testler:** Playwright ile kritik kullanıcı akışları: login, yazı oluşturma, görüntüleme

9.3. Kritik Test Senaryoları

12. XSS payload'u içeren HTML kaydedilince sanitize sonrası temiz çıkmalı
13. Geçersiz JWT ile /api/admin/* endpoint'lerine istek 401 dönmeli
14. Rate limit aşıldınca /api/token/ endpoint'i 429 dönmeli
15. 5MB üzeri dosya yüklemesi reddedilmeli
16. Slug çakışması durumunda POST /api/admin/posts/ 400 dönmeli
17. Refresh token rotasyonu: eski token ikinci kullanımda 401 dönmeli

10. CI/CD Pipeline ve Deploy Stratejisi

10.1. GitHub Actions Pipeline

Her pull request ve main branch push'unda otomatik olarak çalışır.

Adım	Eylemler
1. Lint	ESLint (Frontend) + flake8/ruff (Backend)
2. Test	pytest (Backend) + Vitest (Frontend)
3. Build	React prod build — bundle boyutu kontrolü
4. Docker build	Backend Docker image build ve push (main branch only)
5. Deploy	Frontend: Vercel otomatik deploy / Backend: DigitalOcean App Platform
6. Smoke test	Deploy sonrası /api/posts/ ve /api/projects/ endpoint'leri kontrol edilir

10.2. Docker Yapılandırması

```
# Dockerfile — çok aşamalı build
FROM python:3.12-slim AS base
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
EXPOSE 8000
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000"]
```

10.3. Ortam Değişkenleri

Tüm hassas konfigürasyon ortam değişkenleri üzerinden yönetilir; kaynak koduna asla dahil edilmez.

Değişken	Zorunlu	Açıklama
SECRET_KEY	Evet	Django secret key — min 50 karakter rastgele
DATABASE_URL	Evet	PostgreSQL bağlantı URL'si
ALLOWED_HOSTS	Evet	Virgülle ayrılmış domain listesi
CORS_ALLOWED_ORIGINS	Evet	Frontend domain'leri
AWS_ACCESS_KEY_ID	Prod	S3 erişim anahtarı
AWS_SECRET_ACCESS_KEY	Prod	S3 gizli anahtar
AWS_STORAGE_BUCKET_NAME	Prod	S3 kova adı

AWS_CLOUDFRONT_DOMAIN	Prod	CDN domain'i
REDIS_URL	Opsiyonel	Önbellekleme ve rate limit için

11. Sonuç ve Yol Haritası

Bu teknik mimari, statik site oluşturucuların hızını dinamik içerik yönetim sistemlerinin esnekliğiyle bir araya getirmektedir. v2.0 revizyonu ile birlikte sistem artık medya yönetimi, arama altyapısı, kapsamlı güvenlik önlemleri ve test stratejisiyle production ortamına hazır bir olgunluğa ulaşmıştır.

11.1. Geliştirme Aşamaları

Aşama	Süre (tahmini)	Kapsam
Aşama 1	2-3 hafta	Temel altyapı: Django kurulumu, JWT auth, BlogPost + Project CRUD, temel frontend
Aşama 2	2-3 hafta	GrapesJS entegrasyonu, save pipeline, admin paneli, ikili içerik saklama
Aşama 3	1-2 hafta	Medya yönetimi: S3 entegrasyonu, görsel yükleme, asset manager
Aşama 4	1 hafta	Arama, kategori/etiket sistemi, SEO meta alanları
Aşama 5	1 hafta	Test yazımı, CI/CD pipeline kurulumu, Docker yapılandırması
Aşama 6	1 hafta	Production deploy, DNS konfigürasyonu, performans testleri, smoke testler

11.2. Gelecek Geliştirmeler (Backlog)

- **Yorum sistemi:** Anonim veya kimlik doğrulamalı yorum desteği
- **RSS beslemesi:** Blog yazıları için standart RSS/Atom feed endpoint'i
- **Çok dil desteği:** django-modeltranslation ile TR/EN içerik yönetimi
- **Analitik dashboard:** Basit görüntülenme ve tıklanma istatistikleri — harici araçsız
- **GrapesJS eklentileri:** Kod sözdizimi renklendirme (Prism.js) bloğu, YouTube embed bloğu
- **GraphQL endpoint:** DRF REST API'ye ek olarak strawberry-django ile GraphQL desteği

Mimari Değerlendirme

Bu tasarım, yalnızca bir kişisel portfolyo sitesinin ihtiyaçlarını değil, gerçek bir yazılım mühendisinin mimari düşünce biçimini yansıtmaktadır. Headless mimari, ikili içerik saklama stratejisi, çift katmanlı XSS koruması ve kapsamlı API tasarımı — bu kararların toplamı, sistemi basit bir vitrin olmaktan çıkarıp ölçeklenebilir bir platform seviyesine taşımaktadır.